

Impact of Refactoring Architectural Smells on Quality, Security, and Performance Metrics

Francesca Arcelli Fontana	Mattia Milanese	Francesco Refolli	Catia Trubiani
University of Milano-Bicocca	University of Milano-Bicocca	University of Milano-Bicocca	Gran Sasso Science Institute
Milan, Italy	Milan, Italy	Milan, Italy	L'Aquila, Italy
francesca.arcelli@unimib.it	m.milanese@campus.unimib.it	f.refolli@campus.unimib.it	catia.trubiani@gssi.it

Abstract—Architectural smells have been widely studied in the literature, and some approaches have been developed to assess their impact on software quality metrics. Yet, empirical evidence bridging architectural smells and software quality metrics, while jointly considering security and performance issues, is limited. In this study, we investigate whether refactoring architectural smells can improve software quality and address security and performance issues that frequently occur in open-source projects. To this end, we analyze software code across various application domains and automatically detect architectural smells, as well as security and performance issues, using a diverse set of tools. Then, we manually remove detected instances of architectural smells and check whether their removal affects quality, security, and performance metrics originally collected in the selected projects. Our results indicate that refactoring architectural smells improves software quality and performance indicators, but we observe a less significant impact on security vulnerabilities.

Index Terms—Architectural Smells Refactoring, Security, Performance, Open-source Software, Technical Debt.

I. INTRODUCTION

Software evolution and maintenance play a central role in modern software development processes. Increasing attention has been devoted to improving software quality, particularly in terms of scalability, and long-term sustainability [1]. Among the factors that negatively affect these dimensions, *architectural smells* (AS) are recognized as a major contributor to Technical Debt (TD) [2], making their detection and refactoring essential to improve the software quality [3].

The internal quality of software systems is commonly assessed through well-established metrics such as the Chidamber & Kemerer (CK) suite [4]. For instance, the so-called *Weighted Methods per Class* (WMC) measures the complexity of a class by considering the complexity of its methods (often weighted by cyclomatic complexity), and it has been widely used to identify poorly designed and complex classes. Empirical studies have shown that metrics like Depth of Inheritance Tree (DIT), Coupling Between Objects (CBO), and Lack of Cohesion of Methods (LCOM) are strongly associated with software defects and quality outcomes [5].

Beyond software quality, there are additional system characteristics that warrant attention from software architects. This study proposes extending the analysis to security vulnerabilities and performance problems, as both constitute critical quality dimensions [6]. Security weaknesses identified through

static analysis tools are known to degrade system reliability [7]. Patching vulnerabilities may introduce maintainability drawbacks or increase complexity [8]. Performance issues, such as high CPU usage and memory consumption, further degrade overall software quality [9].

Although architectural smells have been extensively studied, to the best of our knowledge, no prior work has systematically examined their relationships with CK metrics, security vulnerabilities, and performance issues. Regarding performance, a preliminary investigation exists [10], but the evidence remains limited in scope, systems, and evaluation metrics. The objective of this research is to assess whether refactoring selected smells affects multiple quality aspects. We conduct an empirical study on three distinct dimensions: (i) code-related metrics, i.e., software quality expressed in terms of technical debt indexes [11] and CK metrics [4]; (ii) security-related metrics, i.e., software vulnerabilities frequently occurring in open-source projects [12]; (iii) performance-related metrics, i.e., efficiency problems, such as bottlenecks or deadlocks [13], that may hinder the successful development of projects.

A variegated set of open-source projects and architectural smells is considered for this study, selected based on vulnerability reports, test suite availability, and feasibility of repeated execution. Technical debt indexes and CK metrics are extracted using the following tools: ARCAN, DESIGNITE, and UNDERSTAND [14]. Security vulnerabilities are analyzed before and after refactoring using SONARQUBE, QODANA, and FORTIFY [15]. Performance is measured using JPROFILER across ten test-suite executions for five systems. Architectural smells were detected with ARCAN (Dependency smells) and DESIGNITE (Modularity smells). Refactoring was guided by these tools and conducted in accordance with established design principles, such as SOLID principles [16].

Summarizing, this work provides a quantitative assessment of the impact of refactoring architectural smells on software quality, security, and performance. The main contributions of this paper are: (i) the evaluation of software quality, along with security vulnerabilities and performance issues on a representative set of open-source projects; (ii) the connection between refactorings and quality, security, and performance-related metrics, thereby identifying which refactorings show an impact on these dimensions. To support replicability, all refactored code and datasets are publicly available [17].

II. RELATED WORK

Software Quality. Gnoyke et al. [18] present an empirical study of how architectural smells evolve and identify typical patterns they exhibit in 14 open-source systems. One of the main findings is that cyclic dependencies at the class level are prone to increasing their complexity over time, due to smells that may merge, often resulting in tangled multi-hubs. Sas et al. [19] focus on the relationship between architectural smells and co-changes, i.e., files that change concurrently. The study points out that in over 90% of cases, co-changes precede the introduction of smells, hence maintaining low complexity reduces the risk of architectural degradation. Jolak et al. [20] conduct an empirical investigation on the impact of smells on software maintainability. They highlight that these smells can have long-term negative effects on software maintainability and evolvability, underscoring the importance of identifying and managing smells to enhance software quality.

Security Vulnerabilities. Sultana et al. [21] examine the relationship between code smells, architectural smells, and software vulnerabilities. Their study demonstrates that, while some code smells are significantly correlated with software vulnerabilities, there is no strong evidence linking architectural smells to vulnerabilities. Oishwee et al. [22] perform an exploratory study on nine Apache projects showing 126 vulnerabilities. They employ GETSMELLS and PMD to detect code and architectural smells. The results show that some smells have a statistically significant relationship with vulnerabilities. However, the analysis of fix-commits reveals that smells and issues often persist even after addressing vulnerabilities, suggesting correlation but not direct causation. Gigante et al. [23] analyze 27 projects to assess the impact of quality-oriented refactoring on software security. Their findings show that, in most cases, removing code smells detected by SonarQube has the side effect of eliminating vulnerabilities identified by Fortify Static Code Analyzer.

Performance Issues. Zhao et al. [24] present a controlled experiment to evaluate if the granularity of microservice-based applications impacts energy consumption and performance. Three levels of granularity (i.e., coarse, medium, and fine) are considered, and a complex relationship exists between them and performance. Arcelli et al. [10] investigate the impact of refactoring three architectural smells on key software performance metrics, i.e., system response time and resources (e.g., CPU and memory) utilization. They remove 65 instances of two types of smells from two open-source projects, and provide evidence of a decrease not only in execution time, but also in memory consumption. Trubiani et al. [25] automatically detect software performance antipatterns in Java-based applications and observe that antipattern-based refactorings can result in significant improvements, as CPU utilization and system response time decrease substantially.

Summarizing, to the best of our knowledge, there is a gap in exploring the impact of refactoring architectural smells on the selected three dimensions, i.e., software quality, security vulnerabilities, and performance issues.

III. STUDY DESIGN

The primary goal of this study is to investigate the impact of refactoring architectural smells while jointly considering software quality, security, and performance. To this end, we formulate the following research questions:

- **RQ1:** What is the impact of refactoring architectural smells on software quality metrics?

Motivation. Architectural smells often indicate the presence of technical debt and design flaws that can hinder the software quality. The aim is to verify whether the selected refactorings effectively remove these smells and result in measurable improvements of software quality metrics, thus providing empirical evidence of their practical value.

- **RQ2:** What is the impact of refactoring architectural smells on the security vulnerabilities?

Motivation. Poor architectural design can promote unsafe coding practices, such as the promiscuous use of object references or pointers, often arising from cyclic dependencies or cyclic hierarchies smells. This question provides a perspective not extensively explored in prior work, as it investigates whether refactoring architectural smells mitigates these risks, providing evidence on how architectural improvements can directly reduce security vulnerabilities.

- **RQ3:** What is the impact of refactoring architectural smells on the performance issues?

Motivation. Previous studies have analyzed the performance impact of refactorings considering a limited subset of architectural smells, often reporting positive effects. This question extends the analysis to a broader set of smell types, aiming to provide a more comprehensive empirical assessment of how architectural refactoring affects software performance indicators.

These questions guide the subsequent selection of tools, projects, and methodologies. Details are provided hereafter.

A. Tools and Data Collection

Different tools are selected to collect the data.

1) Architectural Smells Detection:

- Arcan¹ is a specialized tool for architectural smells detection and Technical Debt estimation (*Technical Debt Index*), with a well-established track record in empirical software engineering research [26], [19], [14]. Moreover, Arcan provides a visualization of smells through dependency graphs, offering a clear overview of the codebase and identifying specific files that require more attention, particularly useful for refactoring. In this study, Arcan was used to detect only four types of architectural smells: *Cyclic Dependencies* (CD), *Unstable Dependencies* (UD), *Hublike Dependencies* (HD), and *Cyclic Hierarchies* (CH).

¹<https://github.com/Arcan-Tech>

- Designite⁽²⁾ is a static analysis tool developed by Sharma et al. [27]. It detects a wide range of quality issues, including architectural, design, implementation, and testability, and smells that may compromise maintainability. Designite has been selected because it detects smells beyond the dependency graph's topology, capturing issues related to modularity and components' responsibilities. In this study, we use the tool to detect two specific smells: *God Component* (GC), and *Feature Concentration* (FC).

2) Technical Debt and CK Metrics Computation:

- Arcan estimates Technical Debt through the Index computation based on the detected smells, accounting for their severity and lines of code (LOC) size.
- Designite computes a subset of the CK metrics, including *Fan-In* (FI), *Number of Children* (NOC), and *Depth of Inheritance Tree* (DIT).
- Understand⁽³⁾ is a static analysis tool used primarily for maintaining, measuring, and analyzing large codebases. This tool complements the metric catalogue by providing additional CK metrics. For this study, we compute *Sum Cyclomatic Complexity* (WMC), *Lack of Cohesion of Methods* (LCOM), and *Coupling Between Objects* (CBO) metrics.

3) Security Vulnerabilities Detection:

- Qodana⁽⁴⁾ is a static analysis tool capable of detecting a wide range of security vulnerabilities, such as SQL Injection, Insecure Deserialization, and Cross-Site Scripting (XSS), Code Smells, Code Duplications, Compliance checks, and Performance bottlenecks, thereby identifying inefficient code sections that may impact execution.
- SonarQube⁽⁵⁾ is a well-known static code analysis tool that provides comprehensive code quality metrics, including: security vulnerabilities, detecting common security issues and potential threats, code quality metrics, Technical Debt quantifying the effort required to fix code quality issues, Test coverage, and Code Reliability issues.
- Fortify SCA⁽⁶⁾ is a Software as a Service (SaaS) solution that offers advanced application security testing through different types of analyses of vulnerabilities: *Static assessments*, i.e., analyzing the application's source code, bytecode, or binary code; *Dynamic assessments*, i.e., evaluating running web applications; *Mobile assessments*, i.e., inspecting mobile application binaries along with backend and network services; *Open-source composition analyses*, i.e., examining third-party components included in the project.

4) Performance Issues Detection:

JProfiler [28] is a Java profiling tool that combines high-level analytics with low-level Java Virtual Machine (JVM) instrumentation to provide detailed insights into application

Project Name	Version	LOC	Description
Apache ActiveMQ	6.1.6	119.2K	Message Broker
Apache Commons Compress	1.28	24.3K	Compression Library
Apache JackRabbit	2.1.6	132.5K	Java Content Repository
Apache Oozie	5.2.1	58K	Workflow Engine
Apache Roller	6.1.5	35.0K	Blog Engine
OpenMRS-core	2.5.3	55.0K	Medical Record System
Jeromq	4.0.0	17.8K	Message Queue

TABLE I
ANALYZED PROJECTS

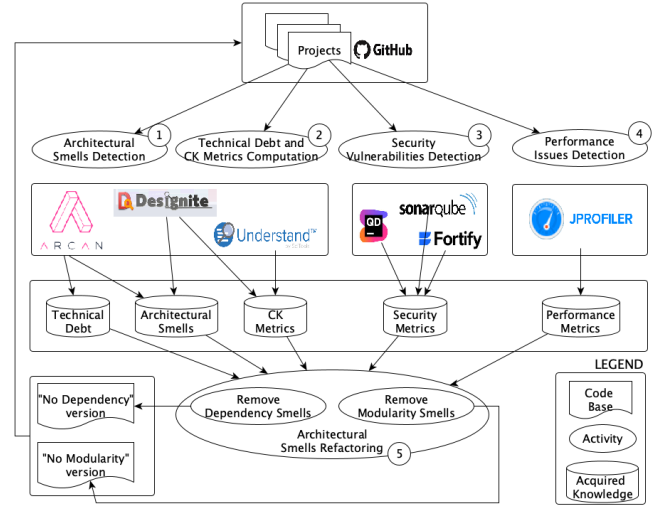


Fig. 1. Pipeline of our Analysis Process

behavior. It is designed to support the detection and diagnosis of performance bottlenecks, memory leaks, thread contention, and other runtime issues, including those related to Java Database Connectivity (JDBC), Java Persistence API (JPA), Java Remote Method Invocation (RMI), and further metrics.

B. Analyzed projects

Table I reports the list of all the analyzed projects showing their name, the considered version, LOC size, and a short description. For this study, RQ1 considers all the projects, whereas RQ2 and RQ3 are formulated on two distinct project groups. This distinction is made because the objective of RQ2 is to identify projects with observable security issues, whereas the purpose of RQ3 is to find projects that include sufficient and functioning test cases for each module, executed within reasonable times to allow proper data collection.

The selected open-source projects for RQ2 are Apache ActiveMQ, Apache Jackrabbit, and Apache Oozie.

The selected open-source projects for RQ3 are: Apache JackRabbit, Apache Roller, OpenMRS-core, Apache Commons Compress, and Jeromq.

C. Analysis Process

Figure 1 illustrates the analysis process that includes the use of the different tools at different stages. The starting point is represented by the code base (extracted from GitHub) in its "Original" version. Projects are split in three groups depending on the research questions. Each step in the diagram

²<https://www.designite-tools.com/docs/designitejava.html>

³<https://scitools.com/>

⁴<https://www.jetbrains.com/qodana/>

⁵<https://www.sonarsource.com/products/sonarqube>

⁶<https://www.opentext.com/en-gb/products/fortify-on-demand/trial>

and its corresponding description is annotated with a number to indicate the sequence of tasks in the process. Specifically:

- ① Each project is analyzed with *Arcan* and *Designite* to detect the architectural smells.
- ② Each project is analyzed with *Arcan*, *Designite*, and *Understand* to estimate Technical Debt, and compute the CK metrics.
- ③ Projects belonging to the RQ2 group are additionally analyzed using SonarQube, Qodana, and Fortify to detect security vulnerabilities.
- ④ Projects in the RQ3 group are profiled with JProfiler to detect performance issues.
- ⑤ For each project, two refactored versions are produced starting from the original codebase. In the *NoDependency* version, dependency-related smells (i.e., CD, UD, HD, CH) are removed, whereas in the *NoModularity* version, modularity-related smells (i.e., FC, GC) are removed. In particular, for the RQ2 group the *NoDependency* and *NoModularity* versions are refactored by removing only those smells affecting the classes or packages where SonarQube, Qodana or Fortify report security issues.

Architectural smells refactoring produces refactored versions of the codebase. We refer to “*NoDependency*” and “*NoModularity*” versions as those code bases in which dependency and modularity smells have been removed. These codebases replace the original versions and are re-analyzed using the same toolset to assess changes in software quality, security vulnerabilities, and performance issues.

IV. REFACTORING OF ARCHITECTURAL SMELLS

This section describes the process of analyzing the architectural smells, i.e., before and after refactoring. We illustrate an example of refactoring the Unstable Dependencies (UD) architectural smell in the JackRabbit project. All other refactorings are publicly available in the replication package [17].

A. Architectural Smells Overview and Refactoring Summary

The initial analysis of smells across all projects shows that the vast majority are *Cyclic Dependencies* (CD) and *Cyclic Hierarchies* (CH). This outcome is consistent with the literature, which reports that CD (and consequently CH) occur frequently in object-oriented systems [14], [29]. Moreover, since all selected projects are implemented in Java, a language strongly rooted in object-oriented paradigms, the prevalence of dependency-related smells (i.e., CD, CH, UD, HD) is not unexpected. Overall, in our projects, dependency-related smells account for nearly 95% of all detected smells, but each type is represented by a substantial number of instances, allowing us to perform a meaningful and sound refactoring analysis on all categories considered in this study. A further inspection of smell metrics computed with *Arcan* reveals that, among all the projects, the mean severity of detected smells is close to zero, and the median is around 1.59. To ensure that the refactoring effort focuses on architecturally relevant issues, we intentionally selected smells with higher severity values.

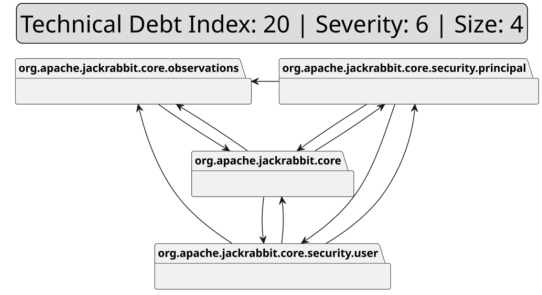


Fig. 2. A view of the Unstable Dependency architectural smell detected with Arcan in the Apache Jackrabbit before Refactoring

This implies that both the mean and median severity of the smells chosen for refactoring are around 4. Targeting higher-severity smells increases the likelihood that the refactoring will have a tangible impact on the architecture, rather than addressing minor issues. Tables II and III summarize the number of removed smells (between parentheses) and those still present after refactoring, accounting for direct removals and indirect removals (i.e., side effects of other refactorings). Across all projects, more than 5k smells were identified, of which nearly 16% were successfully removed (approximately 14% of dependency-related smells and 46% of modularity-related smells). Although the relative percentage of removed dependency-related smells may appear modest, the absolute number is almost one thousand eliminated smells, highlighting the significance of the refactoring effort.

B. An Example of Architectural Smell Refactoring

Hereafter, we describe one example of the Unstable Dependency (UD) smell occurring in the Apache Jackrabbit project. We recall that further examples of refactoring can be found in the replication package [17].

Problem. There exists a package⁷ that has an UD smell with a high Technical Debt Index (20) and severity (6), see Figure 2. The affected component has a size of 4 elements.

Solution. Create a new utility class and place it in a newly created ‘util’ package. Move the functionality that is causing dependencies from the principal component to this new class, thereby removing the direct dependencies from the principal component to other modules.

Design Principles. This refactoring applies the Single Responsibility Principle (SRP) and Dependency Inversion Principle (DIP) [30]. By extracting the functionality causing multiple dependencies into a dedicated utility class, each component maintains a single, well-defined responsibility. The new structure better adheres to the Stable Dependencies Principle by reducing coupling between components.

Impact. The elimination of Unstable Dependencies from the principal component results in a cleaner dependency structure and reduces Technical Debt. The dependency graph shows fewer connections to the principal package, which improves overall architecture stability and makes the system more maintainable and easier to evolve.

⁷See: org.apache.jackrabbit.core.security.principal

Project Name	CD	% Δ CD	CH	% Δ CH	UD	% Δ UD	HD	% Δ HD	Total	% Δ Total
ActiveMQ	658 (-48)	-6.8%	226 (-10)	-4.2%	3 (-2)	-40.0%	10 (-3)	-23.1%	897 (-63)	-6.6%
Oozie	717 (-208)	-22.5%	420 (-2)	-0.5%	3 (-2)	-40.0%	6 (-1)	-14.3%	1146 (-213)	-15.7%
JackRabbit	656 (-43)	-6.2%	276 (-19)	-6.4%	0 (-3)	-100.0%	4 (-5)	-55.6%	936 (-70)	-7.0%
OpenMRS-core	439 (-37)	-7.8%	257 (-0)	0.0%	1 (-2)	-66.7%	7 (-0)	0.0%	704 (-39)	-5.3%
Roller	108 (-19)	-15.0%	19 (-1)	-5.0%	1 (-2)	-66.7%	11 (-0)	0.0%	139 (-22)	-13.7%
Compress	52 (-90)	-63.4%	286 (-1)	-0.3%	1 (-1)	-50.0%	6 (-0)	0.0%	345 (-92)	-21.0%
Jeromq	188 (-200)	-51.5%	140 (-16)	-10.3%	2 (-1)	-33.3%	2 (-0)	0.0%	332 (-217)	-39.5%

TABLE II

ARCHITECTURAL SMELLS REMOVED AFTER REFACTORING (DEPENDENCY-TYPE). THE PROJECT WITH MAXIMUM TOTAL CHANGE IS HIGHLIGHTED.

Project Name	FC	% Δ FC	GC	% Δ GC	Total	% Δ Total
ActiveMQ	61 (-15)	-19.7%	31 (-15)	-32.6%	92 (-30)	-24.6%
Oozie	10 (-19)	-65.5%	7 (-8)	-53.3%	17 (-27)	-61.4%
JackRabbit	58 (-22)	-27.5%	19 (-10)	-34.5%	77 (-32)	-29.4%
OpenMRS-core	8 (-20)	-71.4%	6 (-5)	-45.5%	14 (-25)	-64.1%
Roller	0 (-25)	-100.0%	0 (-4)	-100%	0 (-29)	-100.0%
Compress	3 (-17)	-85.0%	4 (-3)	-42.9%	7 (-20)	-74.1%
Jeromq	4 (-16)	-80.0%	1 (-2)	-66.7%	5 (-18)	-78.3%

TABLE III

ARCHITECTURAL SMELLS REMOVED AFTER REFACTORING (MODULARITY-TYPE). THE PROJECT WITH MAXIMUM TOTAL CHANGE IS HIGHLIGHTED.

Project Name	TD Index	% Δ TD Index	Hours	% Δ Hours
ActiveMQ	10567 (-536)	-4.8%	1923.2 (-97.5)	-4.8%
JackRabbit	6980 (-2585)	-27%	1240.4 (-500.4)	-28.7%
Oozie	6758 (-1187)	-14.94%	1230.0 (-231.4)	-15.84%
OpenMRS-core	5497 (-296)	-5.1%	1000.5 (-53.8)	-5.1%
Roller	1683 (-139)	-7.6%	306.3 (-25.3)	-7.6%
Compress	1966 (-758)	-27.8%	357.8 (-138)	-27.9%
Jeromq	2127 (-1241)	-36.84%	387.1 (-225.9)	-36.9%

TABLE IV

TECHNICAL DEBT DATA COMPUTED BY ARCAN AFTER REFACTORING DEPENDENCY-RELATED SMELLS (CD, CH, UD, HD). THE PROJECT WITH MAXIMUM HOURS DEBT CHANGE IS HIGHLIGHTED.

V. RESULTS

A. Answering RQ1 – Impact on Software Quality Metrics

To answer RQ1, we analyze the impact of refactoring using the Technical Debt Index of Arcan and a set of quality metrics derived mainly from the CK suite and computed using UNDERSTAND and DESIGNITE. Given that architectural smells typically span multiple classes, all metrics are aggregated at the package level. For WMC, package values are obtained by averaging class-level results. For DIT and LCOM, a double averaging process is applied to ensure representativeness, while for CBO, Fan-In, and NOC, class-level values are summed within each package and then averaged. Moreover, the Technical Debt Index is recomputed with Arcan only for the *NoDependency* versions, as the smells removed in the *NoModularity* version are detected with DESIGNITE, which does not provide a Technical Debt estimate nor compute a Technical Debt Index comparable to Arcan.

Table IV shows that, although the number and severity of manually removed smells are broadly comparable across projects, differences in side effects, structural interdependencies, and subsequent adjustments resulted in positive (yet uneven) reductions in the Technical Debt Index. A notable case is Apache JackRabbit, where Technical Debt decreases by more than 27%, a substantially larger improvement than in other systems. This outcome is especially interesting because only about 7% of its dependency-related smells are removed in the *NoDependency* version, a proportion considerably lower than in the other refactored projects. A possible explanation is that the original version of JackRabbit shows one of the highest initial Technical Debt Index values, making it more sensitive to even limited refactoring. This suggests that, in projects with substantial accumulated debt, addressing a relatively small set of architectural issues can still yield disproportionately large gains in maintainability.

Project	Version	WMC (% Δ)	LCOM (% Δ)	CBO (% Δ)
ActiveMQ	NoModularity	148.16 (-52.1%)	10.49 (+41.0%)	28.17 (-8.6%)
	NoDependency	296.97 (-3.9%)	7.49 (+0.8%)	30.74 (-0.3%)
Oozie	NoModularity	153.8795 (-53.0%)	8.3868 (+17.3%)	33.4658 (-6.0%)
	NoDependency	323.43 (-1.2%)	7.15 (-0.1%)	35.17 (-1.2%)
JackRabbit	NoModularity	168.84 (-39.5%)	17.29 (+14.9%)	32.73 (-9.9%)
	NoDependency	275.19 (-1.3%)	15.05 (0.1%)	36.24 (-0.2%)
OpenMRS-core	NoModularity	93.78 (-58.6%)	21.93 (-12.1%)	55.40 (-41.6%)
	NoDependency	214.67 (-5.6%)	24.32 (-2.5%)	92.08 (-2.8%)
Roller	NoModularity	56.80 (-51.2%)	40.97 (+33.7%)	45.62 (-43.6%)
	NoDependency	115.00 (-1.18%)	30.30 (-1.2%)	80.21 (-0.9%)
Compress	NoModularity	173.03 (-40.0%)	21.13 (-15.4%)	47.81 (-4.7%)
	NoDependency	282.11 (-2.2%)	24.78 (-0.8%)	49.75 (-0.8%)
Jeromq	NoModularity	61.38 (-66.1%)	14.07 (-1.9%)	26.32 (-22.5%)
	NoDependency	176.69 (-2.4%)	13.98 (-2.5%)	34.59 (+1.83%)

TABLE V

PACKAGE-LEVEL METRICS FROM UNDERSTAND. SHADED PERCENTAGE VARIATIONS INDICATE IMPROVEMENTS; ALL OTHERS REGRESSIONS.

Project	Version	NOC (% Δ)	DIT (% Δ)	Fan-In (% Δ)
ActiveMQ	NoModularity	4.6102 (-56.3%)	0.2711 (-30.9%)	30.6236 (-53.0%)
	NoDependency	10.1735 (-3.7%)	0.3808 (-2.9%)	62.5799 (-4.0%)
Oozie	NoModularity	3.2484 (-58.2%)	0.1431 (-37.4%)	48.8025 (-54.6%)
	NoDependency	7.6143 (-2.0%)	0.2239 (-2.0%)	105.5714 (-1.8%)
JackRabbit	NoModularity	3.3128 (-44.2%)	0.2413 (-23.4%)	28.1369 (-40.2%)
	NoDependency	5.8486 (-1.53%)	0.3151 (+0.1%)	46.2890 (-1.7%)
OpenMRS-core	NoModularity	1.4904 (-69.0%)	0.0764 (-55.3%)	17.7740 (-60.2%)
	NoDependency	4.4535 (-7.4%)	0.1602 (-6.3%)	42.8721 (-4.0%)
Roller	NoModularity	0.7974 (-57.1%)	0.1444 (-36.7%)	15.1176 (-51.5%)
	NoDependency	1.8028 (-2.9%)	0.2136 (-6.4%)	30.6620 (-1.5%)
Compress	NoModularity	3.1045 (-44.5%)	0.1972 (-14.3%)	24.9254 (-41.9%)
	NoDependency	5.1842 (-7.3%)	0.1937 (-15.8%)	40.8947 (-4.7%)
Jeromq	NoModularity	0.5378 (-64.7%)	0.0655 (-31.8%)	16.2521 (-61.7%)
	NoDependency	1.7250 (+13.1%)	0.1019 (+6.0%)	34.6500 (-18.4%)

TABLE VI

PACKAGE-LEVEL METRICS FROM DESIGNITE. SHADED PERCENTAGE VARIATIONS INDICATE IMPROVEMENTS; ALL OTHERS REGRESSIONS.

Tables V and VI report the CK metrics computed with UNDERSTAND and DESIGNITE after applying the refactorings, together with their percentage change relative to the pre-refactoring values. For the sake of perspective, the tables present the metrics for both the *NoModularity* and *NoDependency* variants of each project. The removal of *modularity-related smells* produces substantial improvements across most CK metrics. Core indicators such as Weighted Methods per Class (WMC) and Coupling Between Objects (CBO) decrease by 50-70%, confirming that modularity-oriented refactorings

effectively reduce structural complexity and increase maintainability. Results for Lack of Cohesion of Methods (LCOM) are more heterogeneous: some systems show improvements, while others exhibit mild degradation. This variability suggests that cohesion is highly sensitive to the specific refactoring actions applied, making it difficult to draw generalizable conclusions.

Refactoring *dependency-related smells* yields smaller yet consistently positive effects. Most projects experience 2 – 5% reductions in their CK metrics, with negative variations (when present) being minor and largely negligible. A notable exception is *Jeromq*, where Fan-In drops sharply (by around 18%), but both Number of Children (NOC) and Depth of Inheritance Tree (DIT) increase (by around 13%). These changes are consistent with the refactoring strategies employed: to eliminate Cyclic Dependencies, Cyclic Hierarchies, and Unstable Dependencies, developers often introduce new abstractions and invert dependencies through inheritance. While effective in breaking dependency cycles, such transformations naturally lengthen inheritance chains (increasing DIT) and broaden class hierarchies (increasing NOC).

By answering **RQ1**, we observe that refactoring architectural smells has a significant positive impact on system maintainability and reduces Technical Debt. Removing dependency-related smells yields a modest but consistent improvement in both the TD Index and CK metrics, even when only a small fraction of smells are removed. Refactoring modularity-related smells produces a substantial improvement across almost all CK metrics (up to 70%), although the effect on LCOM is mixed, varying across projects depending on the specific refactoring.

B. Answering RQ2 – Impact on Security Vulnerabilities

To answer RQ2, we collect data with the three tools described above, i.e., Qodana, SonarQube, and Fortify.

The findings are summarized in Table VII, where we report the number of security vulnerabilities detected in the two refactored versions (*NoDependency* and *NoModularity*), together with their difference w.r.t. the original version.

Security Vulnerabilities with Qodana. The analysis of *Apache JackRabbit* reveals several instances of a specific security issue related to outdated dependencies. This is due to the need to keep dependencies up to date, and implementing robust validation mechanisms is crucial for maintaining security. It is named *Vulnerable Declared Dependencies* and 34 instances are identified. This category includes cases where the project explicitly declares dependencies on components with known vulnerabilities. The same category of vulnerabilities is observed in *Apache ActiveMQ* with 61 instances and *Apache Oozie* with 1036 instances.

Security Vulnerabilities with SonarQube. The tool classifies security risks as Low, Medium, High, Info, and Blocker. It enables the identification of vulnerabilities and categorizes them into distinct groups. For *Apache JackRabbit*, among the vulnerabilities detected in Table VII, SonarQube has identified instances belonging to the following categories: (i) *XML*

Project	Version	SonarQube (Δ)	Qodana (Δ)	Fortify (Δ)
ActiveMQ	NoModularity	85 (0)	61 (0)	5916 (-59)
	NoDependency	85 (0)	61 (0)	5910 (-65)
Jackrabbit	NoModularity	178 (0)	34 (0)	296 (0)
	NoDependency	175 (-3)	34 (0)	296 (0)
Oozie	NoModularity	143 (0)	1036 (0)	778 (0)
	NoDependency	143 (0)	1036 (0)	778 (0)

TABLE VII
SECURITY ISSUES WITH SONARQUBE, QODANA, AND FORTIFY BEFORE AND AFTER REFACTORING

*External Entity (XXE)*⁸ (9 instances), i.e., improper handling of XML input that can lead to XXE attacks; (ii) *Others* (169 instances), i.e., generic issues not matching specific security labels. In the *Apache ActiveMQ* project, the detected categories are: (i) *Object Injection*⁹ (26 instances), i.e., deserializing untrusted data can lead to remote code execution; (ii) *Authentication*¹⁰ (1 instance), i.e., weak or missing authentication mechanisms; (iii) *XML External Entity (XXE)* (1 instance), and *Others* (57 instances). In the *Apache Oozie* project, SonarQube has detected: (i) *XML External Entity (XXE)* (7 instances); (ii) *Weak Cryptography*¹¹ (3 instances), i.e., use of weak or outdated cryptographic algorithms that can compromise data confidentiality and integrity; and (iii) *Others* (133 instances).

Security Vulnerabilities with Fortify. It is a security-focused tool that identifies a very large number of vulnerability categories. Due to their extensive number, they are reported in the replication package only, please refer to [17].

By answering **RQ2**, we observe that refactoring architectural smells has a marginal impact on security vulnerabilities. Removing dependency-related or modularity-related smells does not result in significant changes, even when smells involve classes or packages (in)directly related to security flaws. A slight reduction is observed in *Apache ActiveMQ*, mainly in the *Privacy Violation* category, and this may be due to indirect effects of refactoring, such as reduced coupling and improved modularity. Such a limited impact can be attributed to the nature of most vulnerabilities, which often arise from external dependencies or application-level logic (e.g., input validation, authentication, or outdated components) rather than structural design flaws.

C. Answering RQ3 – Impact on Performance Issues

To answer RQ3, we evaluate the impact of refactorings on software performance metrics. Specifically, we consider: (i) *CPU-related metrics*, i.e., *Process load* (the percentage of CPU resources used by the Java process), and *System load* (the overall CPU usage of the system); (ii) *Memory-related metrics*, i.e., *Committed size* (the total memory reserved by the JVM), *Free size* (the portion of committed memory currently unused), *Used size* (the memory occupied by objects and JVM structures), *Record Object Instance*

⁸https://owasp.org/Top10/A04_2021-Insecure_Design/

⁹https://owasp.org/Top10/A08_2021-Software_and_Data_Integrity_Failures/

¹⁰https://owasp.org/Top10/A07_2021-Identification_and_Authentication_Failures/

¹¹https://owasp.org/Top10/A06_2021-Vulnerable_and_Outdated_Components/

Metric	Version	%Δ Max	%Δ Mean	%Δ Min	%Δ Var
Process load	NoModularity	-3.6%	—	—	-2.0%
	NoDependency	-3.2%	—	—	-2.2%
System load	NoModularity	-5.7%	-3.7%	—	-14.2%
	NoDependency	-1.7%	-2.6%	—	-12.1%
Committed size	NoModularity	—	+1.9%	-2.2%	+1.4%
	NoDependency	—	-2.7%	-3.7%	-6.1%
Free size	NoModularity	-1.8%	-1.8%	+10.3%	-1.9%
	NoDependency	+2.5%	+3.0%	+5.3%	-4.7%
Used size	NoModularity	+1.1%	+1.5%	—	+11.3%
	NoDependency	-2.4%	-1.7%	—	-3.0%
Instance Count	NoModularity	-8.2%	-5.2%	—	-18.2%
	NoDependency	-12.8%	-7.8%	—	-20.4%
Size (bytes)	NoModularity	-3.5%	-4.9%	—	-10.9%
	NoDependency	-15.2%	-10.4%	—	-10.5%

TABLE VIII

PERCENTAGE CHANGE IN THE PERFORMANCE METRICS AFTER REFACTORING. SHADED CELLS INDICATE A POSITIVE EFFECT, “—” MEANS NEGLIGIBLE ($|\Delta| \leq 1\%$), ALL OTHERS A NEGATIVE EFFECT.

Count (the number of live instances of each class at the end of execution), and Record Object Size (the total memory used by live object instances).

As outlined in Section III, the metrics are collected using JProfiler in JVM offline mode during test execution. To reduce variability, 10 test runs are performed for each project, and the metrics are averaged across all runs. Results are summarized in Table VIII, and they highlight common patterns in metric changes after the refactorings. It is worth noting that, overall, the positive effect is more often achieved than the negative effect, which is experienced in only 9 out of 56 experiments. We define as negligible effect all the cases with a percentage value lower than one, and we can observe that this is obtained 14 times. The positive effect is observed 33 times, representing 58% of all experiments.

By answering **RQ3**, we see that refactoring architectural smells generally improves system performance. Regarding memory and object metrics, refactoring dependency and modularity smells reduces the number and size of live object instances, thereby enhancing memory efficiency. However, modularity-driven changes occasionally increase memory usage due to the introduction of new packages and classes. Regarding CPU metrics, both types of refactoring reduce average load and variance, which results in smoother and predictable execution. Overall, while some trade-offs exist, such as increased memory allocation w.r.t. improved modularity, the experiments demonstrate memory efficiency and CPU utilization, confirming tangible performance benefits.

VI. OBSERVATIONS ON THE OBTAINED RESULTS

a) Observations on RQ1 results: by answering RQ1, we notice that the refactoring of the analyzed projects has a positive impact on structural quality in both smell categories. Additionally, we remark that addressing modularization problems has a greater impact on package metrics than addressing dependency problems. This is reinforced by a higher percentage of modularity-related smells removed than dependency-related ones, as a direct consequence of the latter being, in practice, less abundant and easier to remove.

b) Observations on RQ2 results: by answering RQ2, a slight reduction of issues is observed, mainly in the *Privacy*

Violation category. This is likely an indirect effect of refactoring, which may have limited impact on unnecessary exposure of sensitive information. The limited impact can be attributed to the nature of most vulnerabilities, which often deviate from structural design flaws. This outcome is consistent with [21], i.e., there is no direct correlation between architectural smells and vulnerabilities.

c) Observations on RQ3 results: by answering RQ3, we state that refactoring both smell categories has a positive effect on CPU consumption, reducing the average CPU load over time and the variance in most projects. While results are consistent with those reported by Arcelli et al. [10], in this work, we also use overall CPU and JVM-level memory telemetry, and we observe that, in a minority of projects, certain performance metrics deteriorate. Notably, prior work has shown that complex refactorings (particularly those involving modifications to inheritance structures) may even introduce new faults or degrade performance [31].

VII. THREATS TO VALIDITY

External validity. The generalization of the findings is limited by our selection of Java projects: three large Apache projects for security and five medium-sized projects for performance. While the chosen systems are representative of complex, widely used open-source software and include well-established benchmarks, results may not fully extend to other software contexts. The investigation of further programming languages, domains, or proprietary systems is foreseen as part of our future research.

Internal validity. Observed effects depend on the accuracy of detection tools (ARCAN, DESIGNITE) and the applied refactoring. To mitigate threats, only critical true positives smells are refactored, all strategies are documented, and the number of removed smells per category is reported to ensure transparency and reproducibility. Manual refactoring decisions remain a potential source of bias; however, all the refactorings are publicly available for transparency and inspection.

Conclusion validity. For security, the results rely on tool-specific heuristics and detection rules, which may diverge; using multiple tools mitigates this risk. The limited impact on security may also be influenced by the nature of vulnerabilities, many of which are unrelated to internal refactoring. For performance metrics, averages were calculated over ten runs to reduce random variability, although additional repetitions could further enhance robustness. Findings should be interpreted in consideration of these limitations.

Construct validity. Performance metrics (CPU, memory, Record Object statistics) are collected using JPROFILER, while security is assessed with three established static analysis tools. Metrics and tools reflect measurable properties but may not capture all aspects of security or performance. Additionally, results may vary depending on hardware or different tool configurations. Smells are chosen based on tool support; alternative choices could produce different outcomes.

VIII. CONCLUSION AND FUTURE WORK

This paper supports software architects in understanding the connection between refactoring architectural smells and their impact on software quality, security vulnerabilities, and performance issues. The refactoring process applied to the projects focused on fixing a subset of smells, specifically targeting Cyclic Dependency, Cyclic Hierarchy, and Unstable Dependency. Our results lead to the following findings: (i) refactoring efforts effectively reduce Technical Debt, and (ii) improvements are achieved in runtime system performance (with reduced frequency of allocation, memory consumption, and CPU overload), but (iii) there is no significant impact in terms of security vulnerabilities. Nonetheless, no new vulnerabilities are introduced, indicating that the applied refactorings are executed with sufficient rigor and that the residual issues originate from factors other than the smells targeted in this study. These results highlight the need for a more comprehensive approach to security improvements, which may include direct vulnerability mitigation strategies, dependency management, and systematic security testing.

As future research, we are interested in exploring the impact of refactorings targeting additional architectural smells, analyzing a broader set of projects, and increasing the number of addressed instances, thus to better consolidate the possible relationships among smells with respect to software quality, security vulnerabilities, and performance issues.

ACKNOWLEDGMENT

We would like to thank the anonymous reviewers for their valuable feedback. This work has been partially funded by the MUR-PRIN project 20228FT78M DREAM, and MUR Department of Excellence 2023 - 2027 for GSSI.

REFERENCES

- [1] V. Rajlich, "Software evolution and maintenance," in *Future of Software Engineering*, 2014, pp. 133–144.
- [2] P. Rachow and M. Riebisch, "An architecture smell knowledge base for managing architecture technical debt," in *International Conference on Technical Debt*, 2022, pp. 1–10.
- [3] H. Mumtaz, P. Singh, and K. Blincoe, "A systematic mapping study on architectural smells detection," *Journal of Systems and Software*, vol. 173, 2021.
- [4] S. R. Chidamber and C. F. Kemerer, "A metrics suite for object oriented design," *IEEE Trans. on Software Engineering*, vol. 20, no. 6, pp. 476–493, 1994.
- [5] R. Subramanyam and M. S. Krishnan, "Empirical analysis of ck metrics for object-oriented design complexity: Implications for software defects," *IEEE Trans. on Software Engineering*, vol. 29, no. 4, pp. 297–310, 2003.
- [6] A. Alkussayer and W. H. Allen, "A scenario-based framework for the security evaluation of software architecture," in *International Conference on Computer Science and Information Technology*, vol. 5, 2010, pp. 687–695.
- [7] R. A. Khan, S. U. Khan, H. U. Khan, and M. Ilyas, "Systematic literature review on security risks and its practices in secure software development," *IEEE Access*, vol. 10, pp. 5456–5481, 2022.
- [8] S. Reis, R. Abreu, and L. Cruz, "Fixing vulnerabilities potentially hinders maintainability," *Empirical Software Engineering*, vol. 26, no. 6, p. 127, 2021.
- [9] R. Jain, D. Menasce, L. W. Dowdy, V. Almeida, C. U. Smith, and L. G. Williams, *The art of computer systems performance analysis: Techniques*. John Wiley & Sons, 2010.
- [10] F. Arcelli Fontana, M. Camilli, D. Rendina, A. G. Taraboi, and C. Trubiani, "Impact of architectural smells on software performance: an exploratory study," in *International Conference on Evaluation and Assessment in Software Engineering*, 2023, pp. 22–31.
- [11] F. Arcelli Fontana, R. Roveda, and M. Zanoni, "Technical debt indexes provided by tools: A preliminary discussion," in *International Workshop on Managing Technical Debt*, 2016, pp. 28–31.
- [12] G. Prana, A. Sharma, L. K. Shar, D. Foo, A. Santosa, A. Sharma, and D. Lo, "Out of sight, out of mind? how vulnerable dependencies affect open-source projects," *Empirical Software Engineering*, vol. 26, no. 4, p. 59, 2021.
- [13] J. Zhou, S. Silvestro, H. Liu, Y. Cai, and T. Liu, "Undead: Detecting and preventing deadlocks in production software," in *International Conference on Automated Software Engineering*, 2017, pp. 729–740.
- [14] M. Bochicchio, D. Sas, A. Gilardi, and F. Arcelli Fontana, "An empirical study on architectural smells through a pipeline for continuous technical debt assessment," *Information and Software Technology*, vol. 185, p. 107783, 2025.
- [15] K. Tsybulka, "Enhancing code quality through automated refactoring techniques," Ph.D. dissertation, ETSI Informatica, 2024.
- [16] T. Eltaieb, V. Madasu, and T. Venna, "Solid principles in software architecture and introduction to resm concept in oop," *Journal of Engineering Science and Technology*, vol. 2, 2015.
- [17] F. Arcelli Fontana, M. Milanese, F. Refolli, C. Trubiani, "Open Science Artifact: Impact of Refactoring Architectural Smells on Quality, Security, and Performance Metrics," <https://zenodo.org/records/18754704>, 2026.
- [18] P. Gnoyke, S. Schulze, and J. Krüger, "Evolution patterns of software-architecture smells: An empirical study of intra-and inter-version smells," *Journal of Systems and Software*, vol. 217, p. 112170, 2024.
- [19] D. Sas, P. Avgieriou, R. Kruizinga, and R. Scheedler, "Exploring the relation between co-changes and architectural smells," *SN Computer Science*, vol. 2, no. 1, p. 13, 2021.
- [20] R. Jolak, S. Karlsson, and F. Dobsław, "An empirical investigation of the impact of architectural smells on software maintainability," *Journal of Systems and Software*, vol. 225, p. 112382, 2025.
- [21] K. Z. Sultana, Z. Codabux, and B. Williams, "Examining the relationship of code and architectural smells with software vulnerabilities," in *Asia-Pacific Software Engineering Conference*, 2020, pp. 31–40.
- [22] S. J. Oishwee, Z. Codabux, and N. Stakhanova, "An exploratory study on the relationship of smells and design issues with software vulnerabilities," in *International Workshop on Mining Software Repositories Applications for Privacy and Security*, 2022.
- [23] D. Gigante, F. Pecorelli, V. S. Barletta, A. Janes, V. Lenarduzzi, D. Taibi, and M. T. Baldassarre, "Resolving security issues via quality-oriented refactoring: A user study," in *International Conference on Technical Debt*, 2023, pp. 82–91.
- [24] Y. Zhao, T. D. Matteis, and J. Bogner, "How does microservice granularity impact energy consumption and performance? A controlled experiment," in *International Conference on Software Architecture*, 2025, pp. 84–95.
- [25] C. Trubiani, R. Pincirolì, A. Biaggi, and F. Arcelli Fontana, "Automated detection of software performance antipatterns in java-based applications," *IEEE Trans. on Software Engineering*, vol. 49, no. 4, pp. 2873–2891, 2023.
- [26] R. Capilla, F. Arcelli Fontana, T. Mikkonen, P. Bacchiega, and V. Salamanca, "Detecting architecture debt in micro-service open-source projects," in *Euromicro Conference on Software Engineering and Advanced Applications*, 2023, pp. 394–401.
- [27] T. Sharma, P. Mishra, and R. Tiwari, "Designite: A software design quality assessment tool," in *International Workshop on bringing architectural design thinking into developers' daily activities*, 2016, pp. 1–4.
- [28] H. Burchell, O. Larose, S. Kalebá, and S. Marr, "Don't trust your profiler: An empirical study on the precision and accuracy of java profilers," in *International Conference on Managed Programming Languages and Runtimes*, 2023, pp. 100–113.
- [29] H. Melton and E. Tempero, "An empirical study of cycles among classes in java," *Empirical Software Eng.*, vol. 12, no. 4, pp. 389–415, 2007.
- [30] R. C. Martin, *Clean architecture: a craftsman's guide to software structure and design*. Prentice Hall Press, 2017.
- [31] G. Bavota, B. De Carluccio, A. De Lucia, M. Di Penta, R. Oliveto, and O. Strollo, "When does a refactoring induce bugs? an empirical study," in *International Working Conference on Source Code Analysis and Manipulation*, 2012, pp. 104–113.